

TASK 1

```
In [4]: import pandas as pd
import numpy as np
```

```
In [5]: #read and understand the data set
transaction_df = pd.read_excel('QVI_transaction_data.xlsx')
customer_df = pd.read_csv('QVI_purchase_behaviour.csv')

print("--- Data Loading Success ---")
print("Transaction Data Shape:", transaction_df.shape)
print("Customer Data Shape:", customer_df.shape)
```

```
--- Data Loading Success ---
Transaction Data Shape: (264836, 8)
Customer Data Shape: (72637, 3)
```

```
In [6]: transaction_df = transaction_df[transaction_df['LYLTY_CARD_NBR'] != 226000]

print("\n--- Outlier Removed ---")
print("New max of PROD_QTY:", transaction_df['PROD_QTY'].max())
```

```
--- Outlier Removed ---
New max of PROD_QTY: 5
```

```
In [7]: transaction_df = transaction_df[~transaction_df['PROD_NAME'].str.lower().str.contai

print("\n--- Product Filtering Complete ---")
print("Dataset rows after filtering:", transaction_df.shape[0])
```

```
--- Product Filtering Complete ---
Dataset rows after filtering: 245302
```

```
In [8]: unique_dates_count = transaction_df['DATE'].nunique()
print("\n--- Date Check ---")
print("Number of unique days in the data:", unique_dates_count)
```

```
--- Date Check ---
Number of unique days in the data: 364
```

```
In [9]: #foring the conversion of DATE using excel format
transaction_df['DATE'] = pd.to_datetime(transaction_df['DATE'], unit='D', origin='1

print("Actual Data Start Date:", transaction_df['DATE'].min())
print("Actual Data End Date:", transaction_df['DATE'].max())
```

```
Actual Data Start Date: 2018-07-01 00:00:00
Actual Data End Date: 2019-06-30 00:00:00
```

```
In [10]: #finding the missing date
all_dates = pd.date_range(start=transaction_df['DATE'].min(), end=transaction_df['D
missing_date = all_dates[~all_dates.isin(transaction_df['DATE'])]
print('the missing date is:', missing_date)
```

```
the missing date is: DatetimeIndex(['2018-12-25'], dtype='datetime64[ns]', freq='D')
```

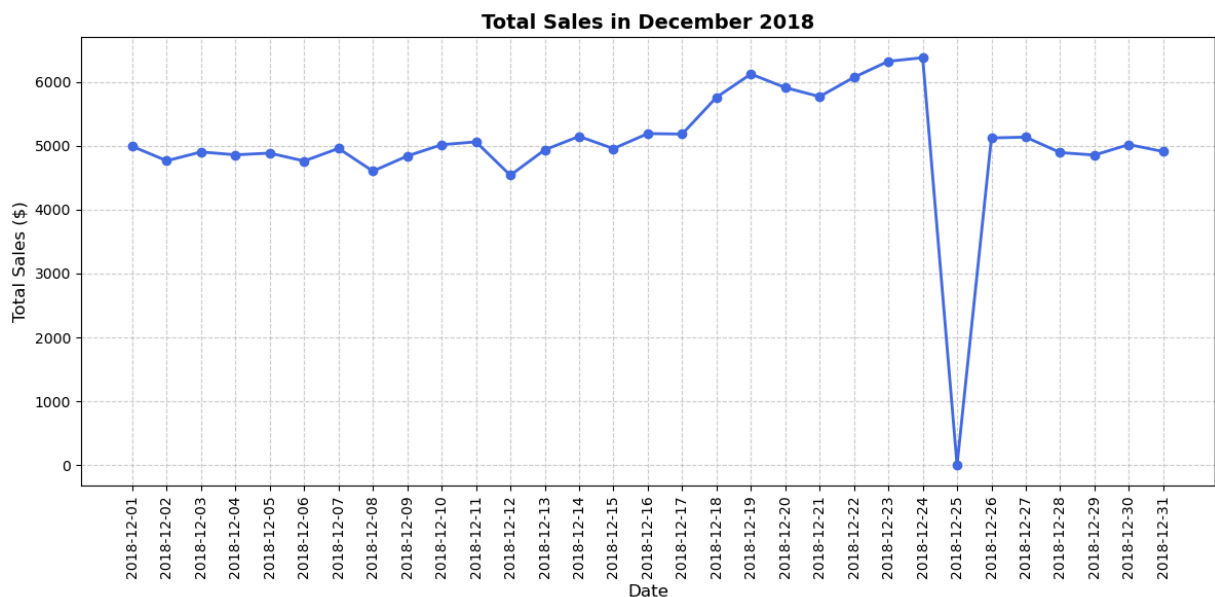
```
In [11]: import matplotlib.pyplot as plt

# Filter data for December 2018 only
december_data = transaction_df[transaction_df['DATE'].dt.month == 12]

# Group by date and calculate total sales for each day
december_sales = december_data.groupby('DATE')['TOT_SALES'].sum().reset_index()

# Create a complete timeline for December to include the missing Christmas day with
dec_timeline = pd.DataFrame({'DATE': pd.date_range(start='2018-12-01', end='2018-12-31')})
december_sales = pd.merge(dec_timeline, december_sales, on='DATE', how='left').fillna(0)

# Plot the line chart
plt.figure(figsize=(12, 6))
plt.plot(december_sales['DATE'], december_sales['TOT_SALES'], marker='o', color='red')
plt.title('Total Sales in December 2018', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Total Sales ($)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xticks(december_sales['DATE'], rotation=90)
plt.tight_layout()
plt.show()
```



Feature Engineering

```
In [12]: # extract the pack size
transaction_df['PACK_SIZE'] = transaction_df['PROD_NAME'].str.extract(r'(\d+)').astype(int)
print("Minimum Pack Size:", transaction_df['PACK_SIZE'].min(), "g")
print("Maximum Pack Size:", transaction_df['PACK_SIZE'].max(), "g")
```

Minimum Pack Size: 70.0 g
Maximum Pack Size: 380.0 g

```
In [13]: #extract brand name
transaction_df['BRAND'] = transaction_df['PROD_NAME'].str.split().str[0]
```

```
#extract the unique brand name
print(transaction_df['BRAND'].unique())
```

```
['Natural' 'CCs' 'Smiths' 'Kettle' 'Grain' 'Doritos' 'Twisties' 'WW'
 'Thins' 'Burger' 'NCC' 'Cheezels' 'Infzns' 'Red' 'Pringles' 'Dorito'
 'Infuzions' 'Smith' 'GrnWves' 'Tyrrells' 'Cobs' 'French' 'RRD' 'Tostitos'
 'Cheetos' 'Woolworths' 'Snbts' 'Sunbites']
```

```
In [14]: #make the brands have the same name
brand_mapping = {
    'Red': 'Red Rock Deli',
    'RRD': 'Red Rock Deli',
    'Dorito': 'Doritos',
    'Smith': 'Smiths',
    'Infzns': 'Infuzions',
    'GrnWves': 'Grain Waves',
    'Grain': 'Grain Waves',
    'WW': 'Woolworths',
    'NCC': 'Natural Chip Co',
    'Natural': 'Natural Chip Co',
    'Snbts': 'Sunbites'
}
transaction_df['BRAND'] = transaction_df['BRAND'].replace(brand_mapping)
print(transaction_df['BRAND'].unique())
```

```
['Natural Chip Co' 'CCs' 'Smiths' 'Kettle' 'Grain Waves' 'Doritos'
 'Twisties' 'Woolworths' 'Thins' 'Burger' 'Cheezels' 'Infuzions'
 'Red Rock Deli' 'Pringles' 'Tyrrells' 'Cobs' 'French' 'Tostitos'
 'Cheetos' 'Sunbites']
```

Profiling purchase behavior data

```
In [15]: print("print the first 5 rows:")
display(customer_df.head())
print('-----')
print("the data set info:")
display(customer_df.info())
print('-----')

print("unique lifestages:")
print(customer_df['LIFESTAGE'].value_counts())
print('-----')

print("Premium Customers:")
print(customer_df['PREMIUM_CUSTOMER'].value_counts())
```

print the first 5 rows:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

the data set info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 72637 entries, 0 to 72636

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	LYLTY_CARD_NBR	72637 non-null	int64
1	LIFESTAGE	72637 non-null	object
2	PREMIUM_CUSTOMER	72637 non-null	object

dtypes: int64(1), object(2)

memory usage: 1.7+ MB

None

unique lifestages:

LIFESTAGE

RETIREEES 14805

OLDER SINGLES/COUPLES 14609

YOUNG SINGLES/COUPLES 14441

OLDER FAMILIES 9780

YOUNG FAMILIES 9178

MIDAGE SINGLES/COUPLES 7275

NEW FAMILIES 2549

Name: count, dtype: int64

Premium Customers:

PREMIUM_CUSTOMER

Mainstream 29245

Budget 24470

Premium 18922

Name: count, dtype: int64

```
In [16]: #merge the two datasets using the LYLTY_CARD_NBR column
merged_df = pd.merge(transaction_df, customer_df ,on='LYLTY_CARD_NBR', how='left')

#check the merged dataset
print(merged_df.shape)
print('-----')
#display a sample of the merged data set
display(merged_df.head())
```

(245302, 12)

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	1
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	



```
In [17]: # Group by Lifestage and Premium Customer to calculate total sales
sales_breakdown = merged_df.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES']

# Sort the results by total sales in descending order
sales_breakdown = sales_breakdown.sort_values(by='TOT_SALES', ascending=False)

# Display the top 10 segments contributing to sales
print("Top 10 Customer Segments by Total Sales:")
display(sales_breakdown.head(10))
```

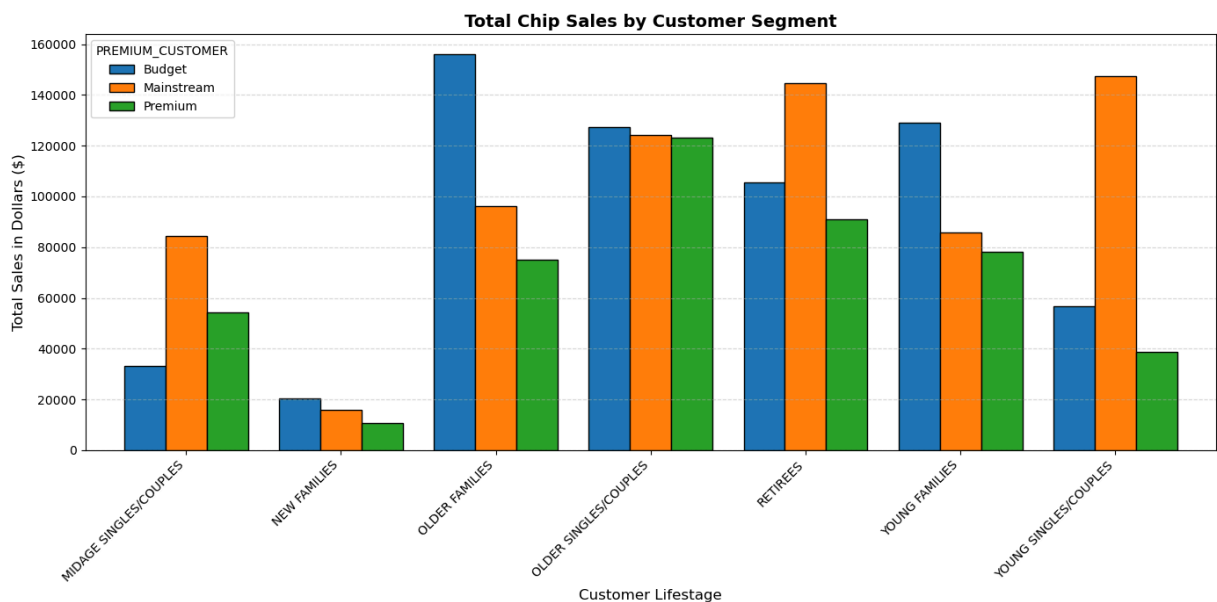
Top 10 Customer Segments by Total Sales:

	LIFESTAGE	PREMIUM_CUSTOMER	TOT_SALES
6	OLDER FAMILIES	Budget	156096.75
19	YOUNG SINGLES/COUPLES	Mainstream	147244.20
13	RETIREEES	Mainstream	144677.55
15	YOUNG FAMILIES	Budget	129151.15
9	OLDER SINGLES/COUPLES	Budget	127279.80
10	OLDER SINGLES/COUPLES	Mainstream	124089.50
11	OLDER SINGLES/COUPLES	Premium	123147.55
12	RETIREEES	Budget	105586.10
7	OLDER FAMILIES	Mainstream	96059.95
14	RETIREEES	Premium	91013.25

```
In [18]: import seaborn as sns
import matplotlib.pyplot as plt

# Pivot the data for clean plotting
sales_pivot = sales_breakdown.pivot(index='LIFESTAGE', columns='PREMIUM_CUSTOMER',

# Plot a grouped bar chart
sales_pivot.plot(kind='bar', figsize=(14, 7), width=0.8, edgecolor='black')
plt.title('Total Chip Sales by Customer Segment', fontsize=14, fontweight='bold')
plt.xlabel('Customer Lifestage', fontsize=12)
plt.ylabel('Total Sales in Dollars ($)', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Dive Deep Analysis

```
In [19]: #No. of unique customer in each segment
customer_counts = merged_df.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].count()

#average no of packets bought by the customer
total_packets_per_user = merged_df.groupby(['LYLTY_CARD_NBR', 'LIFESTAGE', 'PREMIUM_CUSTOMER']).sum()
avg_packets_per_segment = total_packets_per_user.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).mean()

# Merge these metrics together
segment_analysis = pd.merge(customer_counts, avg_packets_per_segment, on=['LIFESTAGE', 'PREMIUM_CUSTOMER'])

# Sort by Unique Customers to see who dominates the store population
segment_analysis = segment_analysis.sort_values(by='UNIQUE_CUSTOMERS', ascending=False)

print("--- Customer Population & Quantity Analysis ---")
display(segment_analysis.head(10))
```

--- Customer Population & Quantity Analysis ---

	LIFESTAGE	PREMIUM_CUSTOMER	UNIQUE_CUSTOMERS	AVG_PACKETS_PER_CUST
19	YOUNG SINGLES/COUPLES	Mainstream	7908	4.564365
13	RETIREEES	Mainstream	6345	5.908274
10	OLDER SINGLES/COUPLES	Mainstream	4854	6.673259
9	OLDER SINGLES/COUPLES	Budget	4839	6.751395
11	OLDER SINGLES/COUPLES	Premium	4679	6.741825
6	OLDER FAMILIES	Budget	4606	9.022579
12	RETIREEES	Budget	4376	6.125457
15	YOUNG FAMILIES	Budget	3951	8.672235
14	RETIREEES	Premium	3808	6.081145
18	YOUNG SINGLES/COUPLES	Budget	3632	4.234306

```
In [20]: #average price/packet
# Calculate the price per unit for each transaction
merged_df['PRICE_PER_UNIT'] = merged_df['TOT_SALES'] / merged_df['PROD_QTY']

# Calculate the average price per unit for each customer segment
avg_price_segment = merged_df.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PRICE_PER

# Sort to see who pays the most on average per packet
avg_price_segment = avg_price_segment.sort_values(by='AVG_PRICE_PER_PACKET', ascend

print("--- Average Price Paid per Packet ---")
display(avg_price_segment.head(10))

--- Average Price Paid per Packet ---
```

	LIFESTAGE	PREMIUM_CUSTOMER	AVG_PRICE_PER_PACKET
19	YOUNG SINGLES/COUPLES	Mainstream	4.071061
1	MIDAGE SINGLES/COUPLES	Mainstream	4.000172
12	RETIREEES	Budget	3.930860
14	RETIREEES	Premium	3.927343
3	NEW FAMILIES	Budget	3.925196
4	NEW FAMILIES	Mainstream	3.921576
11	OLDER SINGLES/COUPLES	Premium	3.899460
9	OLDER SINGLES/COUPLES	Budget	3.890437
5	NEW FAMILIES	Premium	3.877260
13	RETIREEES	Mainstream	3.850556

T-TEST

```
In [21]: from scipy.stats import ttest_ind

# 1. Get prices for Young Singles/Couples - Mainstream
mainstream_prices = merged_df[
    (merged_df['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
    (merged_df['PREMIUM_CUSTOMER'] == 'Mainstream')
]['PRICE_PER_UNIT']

# 2. Get prices for Young Singles/Couples - Budget and Premium
non_mainstream_prices = merged_df[
    (merged_df['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
    (merged_df['PREMIUM_CUSTOMER'].isin(['Budget', 'Premium']))
]['PRICE_PER_UNIT']

# 3. Perform the independent t-test
t_stat, p_value = ttest_ind(mainstream_prices, non_mainstream_prices, alternative='

print("--- Statistical T-Test Results ---")
print(f"T-Statistic: {t_stat}")
print(f"P-Value: {p_value}")

--- Statistical T-Test Results ---
T-Statistic: 35.035080795315
P-Value: 1.7890477228791918e-264
```

```
In [22]: #fav brand for YOUNG SINGLES/COUPLES
# Target segment: Young Singles/Couples - Mainstream
target_segment = merged_df[(merged_df['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (m
other_segments = merged_df[~((merged_df['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &

# Calculate brand proportions for both groups
target_brands = target_segment['BRAND'].value_counts(normalize=True).reset_index(na
other_brands = other_segments['BRAND'].value_counts(normalize=True).reset_index(nam
```



```
# Merge and calculate the affinity affinity_ratio (Target / Other)
brand_affinity = pd.merge(target_brands, other_brands, on='BRAND')
brand_affinity['Affinity_Index'] = brand_affinity['Target_Prop'] / brand_affinity['Other_Prop']

print("--- Top 5 Brands Preferred by Young Mainstream ---")
display(brand_affinity.sort_values(by='Affinity_Index', ascending=False).head(5))
```

--- Top 5 Brands Preferred by Young Mainstream ---

	BRAND	Target_Prop	Other_Prop	Affinity_Index
11	Tyrrells	0.031789	0.025785	1.232863
6	Twisties	0.046220	0.037878	1.220237
1	Doritos	0.122175	0.101160	1.207743
7	Tostitos	0.045707	0.037998	1.202882
0	Kettle	0.197412	0.165806	1.190617

```
In [23]: #calculate the size proportions compared to the other group
# Calculate pack size proportions for both groups
target_sizes = target_segment['PACK_SIZE'].value_counts(normalize=True).reset_index()
other_sizes = other_segments['PACK_SIZE'].value_counts(normalize=True).reset_index()

# Merge and calculate affinity
size_affinity = pd.merge(target_sizes, other_sizes, on='PACK_SIZE')
size_affinity['Affinity_Index'] = size_affinity['Target_Prop'] / size_affinity['Other_Prop']

print("\n--- Top 5 Pack Sizes Preferred by Young Mainstream ---")
display(size_affinity.sort_values(by='Affinity_Index', ascending=False).head(5))
```

--- Top 5 Pack Sizes Preferred by Young Mainstream ---

	PACK_SIZE	Target_Prop	Other_Prop	Affinity_Index
8	270.0	0.031841	0.025085	1.269296
7	380.0	0.032149	0.025639	1.253911
5	330.0	0.061370	0.050237	1.221615
2	134.0	0.118889	0.100903	1.178243
9	210.0	0.029581	0.025223	1.172799

TASK 2 : Experimentation and uplift testing

```
In [27]: merged_df['DATE'] = pd.to_datetime(merged_df['DATE'])
merged_df['YEARMONTH'] = merged_df['DATE'].dt.year * 100 + merged_df['DATE'].dt.month
```

```
In [30]: # AGGREGATE MONTHLY METRICS PER STORE
monthly_store_metrics = merged_df.groupby(['STORE_NBR', 'YEARMONTH']).agg(
    totSales=('TOT_SALES', 'sum'),
    nCustomers=('LYLTY_CARD_NBR', 'nunique'),
```

```

nTxns=('TXN_ID', 'nunique'),
totUnits=('PROD_QTY', 'sum')
).reset_index()

# FILTER THE PR TRAIL DATE
pre_trail_date = monthly_store_metrics[monthly_store_metrics['YEARMONTH'] < 201902]

# keep only the stores that have data for all months in the pre-trial period
store_counts = pre_trail_date.groupby('STORE_NBR')['YEARMONTH'].count()
valid_stores = store_counts[store_counts == 7].index
pre_trail_date = pre_trail_date[pre_trail_date['STORE_NBR'].isin(valid_stores)]

print(f" Pre_trail store are ready Total qualified stores: {len(valid_stores)}")
display(pre_trail_date.head())

```

Pre_trail store are ready Total qualified stores: 260

	STORE_NBR	YEARMONTH	totSales	nCustomers	nTxns	totUnits
0	1	201807	188.9	47	49	58
1	1	201808	168.4	41	41	52
2	1	201809	268.1	57	59	71
3	1	201810	175.4	39	40	51
4	1	201811	184.8	44	45	55

In [31]: `from scipy.stats import pearsonr`

```

def calculate_correlation(df, metric_col, store1, store2):
    """Calculate the Pearson correlation coefficient for a specific metric between
    series1 = df[df['STORE_NBR'] == store1][metric_col].values
    series2 = df[df['STORE_NBR'] == store2][metric_col].values
    corr, _ = pearsonr(series1, series2)
    return corr

def calculate_distance(df, metric_col, store1, store2):
    """Calculate the average absolute distance for a specific metric between two st
    series1 = df[df['STORE_NBR'] == store1][metric_col].values
    series2 = df[df['STORE_NBR'] == store2][metric_col].values

    # Calculate mean absolute error/distance
    dist = np.mean(np.abs(series1 - series2))
    return dist

def find_control_store(trial_store, df):
    """Find the best matching control store for a given trial store."""
    stores = df['STORE_NBR'].unique()
    results = []

    for control_candidate in stores:
        # Avoid comparing the store against itself or other trial stores
        if control_candidate in [trial_store, 77, 86, 88]:
            continue

```

```

# 1. Calculate correlation and distance for total sales (totSales)
corr_sales = calculate_correlation(df, 'totSales', trial_store, control_candidate)
dist_sales = calculate_distance(df, 'totSales', trial_store, control_candidate)

# 2. Calculate correlation and distance for customer count (nCustomers)
corr_cust = calculate_correlation(df, 'nCustomers', trial_store, control_candidate)
dist_cust = calculate_distance(df, 'nCustomers', trial_store, control_candidate)

results.append({
    'control_store': control_candidate,
    'corr_sales': corr_sales,
    'dist_sales': dist_sales,
    'corr_cust': corr_cust,
    'dist_cust': dist_cust
})

res_df = pd.DataFrame(results)

# Normalize distances into similarity scores (Normalized Distance Score: 1 - min)
res_df['dist_sales_score'] = 1 - ((res_df['dist_sales'] - res_df['dist_sales'].min()) / (res_df['dist_sales'].max() - res_df['dist_sales'].min()))
res_df['dist_cust_score'] = 1 - ((res_df['dist_cust'] - res_df['dist_cust'].min()) / (res_df['dist_cust'].max() - res_df['dist_cust'].min()))

# Calculate final composite score (average of all metrics)
res_df['final_score'] = (res_df['corr_sales'] + res_df['corr_cust'] + res_df['dist_sales_score'] + res_df['dist_cust_score']) / 4

# Sort descending to pick the store with the highest score
best_control = res_df.sort_values(by='final_score', ascending=False).iloc[0]['control_store']

return int(best_control), res_df

# Run matching for trial stores (77, 86, 88)
trial_stores = [77, 86, 88]
control_matches = {}

for t_store in trial_stores:
    c_store, _ = find_control_store(t_store, pre_trial_date)
    control_matches[t_store] = c_store
    print(f"🌀 Trial Store {t_store} ---> Matched Control Store: {c_store}")

🌀 Trial Store 77 ---> Matched Control Store: 233
🌀 Trial Store 86 ---> Matched Control Store: 155
🌀 Trial Store 88 ---> Matched Control Store: 237

```

In [32]: `from scipy.stats import t`

```

# Define trial period (Feb 2019 to Apr 2019)
trial_months = [201902, 201903, 201904]

# Function to assess trial store vs control store performance
def assess_trial_store(trial_store, control_store, df_all, metric='totSales'):
    # 1. Pre-trial ratio calculation
    pre_trial_df = df_all[df_all['YEARMONTH'] < 201902]

    trial_pre_mean = pre_trial_df[pre_trial_df['STORE_NBR'] == trial_store][metric].mean()
    control_pre_mean = pre_trial_df[pre_trial_df['STORE_NBR'] == control_store][metric].mean()

```

```

scaling_factor = trial_pre_mean / control_pre_mean

# 2. Get trial period data
trial_data = df_all[(df_all['YEARMONTH'].isin(trial_months)) & (df_all['STORE_NBR'].isin(trial_stores))]
control_data = df_all[(df_all['YEARMONTH'].isin(trial_months)) & (df_all['STORE_NBR'].isin(control_stores))]

# Scale control metrics
control_data['scaled_metric'] = control_data[metric] * scaling_factor

# Merge performance
comparison = pd.merge(
    trial_data[['YEARMONTH', metric]],
    control_data[['YEARMONTH', 'scaled_metric']],
    on='YEARMONTH'
)

comparison['percentage_diff'] = ((comparison[metric] - comparison['scaled_metric']) / comparison[metric]) * 100

# 3. Calculate std deviation of pre-trial difference for significance test
pre_trial_merged = pd.merge(
    pre_trial_df[pre_trial_df['STORE_NBR'] == trial_store][['YEARMONTH', metric]],
    pre_trial_df[pre_trial_df['STORE_NBR'] == control_store][['YEARMONTH', metric]],
    on='YEARMONTH',
    suffixes=('_trial', '_control')
)
pre_trial_merged['scaled_control'] = pre_trial_merged[f'{metric}_control'] * scaling_factor
pre_trial_merged['diff'] = pre_trial_merged[f'{metric}_trial'] - pre_trial_merged['scaled_control']

std_dev = pre_trial_merged['diff'].std()
degrees_of_freedom = 7 - 1 # 7 pre-trial months

# t-value for 95% confidence interval
t_critical = t.ppf(0.975, df=degrees_of_freedom)

comparison['t_value'] = (comparison[metric] - comparison['scaled_metric']) / std_dev
comparison['significant'] = comparison['t_value'] > t_critical

return comparison

# Function to plot trend comparisons
def plot_trial_comparison(trial_store, control_store, df_all, metric='totSales'):
    pre_trial_df = df_all[df_all['YEARMONTH'] < 201902]
    scaling_factor = (pre_trial_df[pre_trial_df['STORE_NBR'] == trial_store][metric].mean() /
                     pre_trial_df[pre_trial_df['STORE_NBR'] == control_store][metric].mean())

    plot_df = df_all[df_all['STORE_NBR'].isin([trial_store, control_store])].copy()

    # Create scaled control metric for entire timeline
    plot_df['plot_metric'] = np.where(
        plot_df['STORE_NBR'] == control_store,
        plot_df[metric] * scaling_factor,
        plot_df[metric]
    )

    plot_df['STORE_TYPE'] = np.where(
        plot_df['STORE_NBR'] == trial_store,
        'Trial',
        'Control'
    )

```

```

        f'Trial Store {trial_store}',
        f'Control Store {control_store} (Scaled)'
    )

    plt.figure(figsize=(10, 5))
    sns.lineplot(data=plot_df, x='YEARMONTH', y='plot_metric', hue='STORE_TYPE', ma
    plt.axvspan(201902, 201904, color='yellow', alpha=0.3, label='Trial Period')
    plt.title(f'Performance Comparison: Trial Store {trial_store} vs Control Store
    plt.xlabel('Year-Month')
    plt.ylabel(metric)
    plt.xticks(rotation=45)
    plt.legend()
    plt.tight_layout()
    plt.savefig(f'Trial_Store_{trial_store}_{metric}.png')
    plt.show()

# Run evaluation for all store pairs
for t_store, c_store in control_matches.items():
    print(f"\n===== Assessment for Trial Store {t_store} vs Control S
    sales_eval = assess_trial_store(t_store, c_store, monthly_store_metrics, metric
    display(sales_eval)

# Generate visualization
plot_trial_comparison(t_store, c_store, monthly_store_metrics, metric='totSales

```

```

===== Assessment for Trial Store 77 vs Control Store 233 =====
=====

```

	YEARMONTH	totSales	scaled_metric	percentage_diff	t_value	significant
0	201902	211.6	231.887625	-8.748904	-0.858098	False
1	201903	255.1	187.023096	36.400266	2.879425	True
2	201904	258.1	151.509721	70.352105	4.508411	True

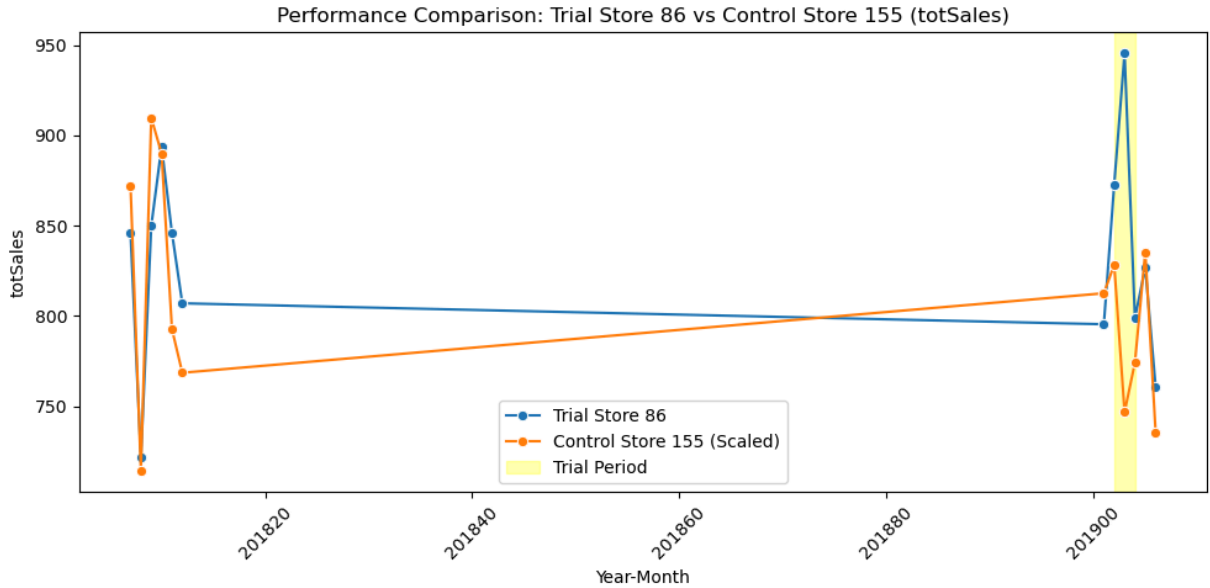


```

===== Assessment for Trial Store 86 vs Control Store 155 =====
=====

```

	YEARMONTH	totSales	scaled_metric	percentage_diff	t_value	significant
0	201902	872.8	828.327259	5.368982	1.151576	False
1	201903	945.4	746.740724	26.603514	5.144076	True
2	201904	798.8	774.195858	3.178025	0.637099	False



===== Assessment for Trial Store 88 vs Control Store 237 =====
=====

	YEARMONTH	totSales	scaled_metric	percentage_diff	t_value	significant
0	201902	1339.6	1300.879003	2.976526	0.401465	False
1	201903	1467.0	1166.728952	25.736144	3.113258	True
2	201904	1317.0	1142.950509	15.228086	1.804573	False



Quantum Retail Strategy & Analytics - End-to-End Analysis

Executive Summary

Brief 3-4 sentence overview of key findings from both tasks and the final recommendation for the Category Manager.

Task 1: Data Preparation & Exploratory Data Analysis

1.1 Data Cleaning & Preprocessing

- Outlier removal (e.g., commercial customer card #226000).
- Date formatting & missing date imputation (Christmas Day closure analysis).
- Feature engineering (extracting `PACK_SIZE` and `BRAND`).

1.2 Customer Segment Insights

- Total sales & transaction analysis by `LIFESTAGE` and `PREMIUM_CUSTOMER` .
 - Key findings: Mainstream Young Singles/Couples buying habits & price willingness (t-test validation).
 - Brand Affinity & Pack Size preferences (e.g., Tyrrells and 270g Twisties packs).
-

Task 2: Experimentation & Uplift Testing

2.1 Control Store Selection

- Pre-trial metrics aggregation (July 2018 – January 2019).
- Similarity matching algorithm (Pearson Correlation + Normalized Distance Score).
- Matched pairs:
 - Trial 77 ↔ Control 233
 - Trial 86 ↔ Control 155
 - Trial 88 ↔ Control 237

2.2 Assessment of Trial Performance

- Control scaling & monthly uplift calculation during trial months (Feb 2019 – Apr 2019).
 - Statistical significance testing (t-test / 95% confidence intervals).
 - Performance trend visualizations.
-

Final Business Recommendations

- Strategic layout rollout decision for Julia and the Category Manager.